Limbo 실물 로봇 초기 구축 및 첫 주행 공정

작성일: 2026-10-20 · 상태: 실행 계획 (실물에서 아직 검증하지 않음)

이 문서는 시뮬레이션에서 주행한 Limbo를 Jetson Orin Nano Super 개발자 키트 + Livox MID-360S + Luxonis OAK-D Lite FF + RoboClaw 기반 2WD로 옮기는 순서다. 실제 TF의 프레임 배치는 Gazebo와 동일하다는 전제를 사용한다. ToF 센서는 첫 주행에서 제외한다. 케이블의 핀 연결과 전원 분배는 하드웨어 담당자가 실물 부품의 정격과 커넥터를 확인해 확정한다.

> 현재 저장소에는 실물 RoboClaw 구동/엔코더 드라이버와 실물 전용 통합 launch가 없다. 아래 명령을 모두 실행한다고 바로 주행하는 상태는 아니다. 7단계의 구현이 완료되어야 한다.

전체 순서와 통과 기준

순서	작업	다음 단계로 넘어갈 기준
0	장비와 파라미터 기록	보드/저장장치/전원/모터/엔코더/센서 모델과 연결 방식 확인
1	Jetson 펌웨어와 Ubuntu 설치	Ubuntu 24.04, aarch64, JetPack/L4T 버전 확인
2	ROS Jazzy와 Limbo 빌드	Jetson에서 필요한 ROS 패키지와 자체 패키지가 빌드됨
3	전원 · USB · Ethernet 연결	각 장치를 단독으로 켜고 Jetson에서 인식 확인
4	RoboClaw 저속 단독 시험	바퀴를 띄운 상태에서 방향 · 엔코더 · 정지 동작 확인
5	MID-360S 단독 시험	실제 PointCloud2가 수신되고 프레임 · 시간 확인
6	OAK 단독 시험	영상 · 정렬된 깊이 · CameraInfo · 포인트클라우드 확인
7	Limbo의 실물용 인터페이스 구현	/cmd_vel→모터, 엔코더→/odom, 센서→기존 주행 입력 연결
8	TF · 시간 · 지도 · Nav2 통합	각 데이터의 frame/stamp와 위치 추정 확인
9	제한된 공간에서 첫 주행	저속 수동 전진/회전, 정지, 자율 목표 순으로 통과

각 단계의 통과 기준을 충족하기 전에는 다음 단계의 모터 구동을 하지 않는다. 첫 주행에는 사람이 즉시 누를 수 있는 물리적 비상정지가 필요하다.

0. 먼저 기록할 값

아래 표를 팀 공유 문서에 실제 값으로 채운다. 값이 없으면 소프트웨어 파라미터를 확정할 수 없다.

항목	기록할 실제 값과 이유
Jetson	개발자 키트/캐리어 보드 모델, 저장장치(microSD 또는 NVMe), 전원 어댑터, 펌웨어 · JetPack 버전. 설치 이미지와 핀/전원 조건이 달라진다.
LiDAR	MID-360S 시리얼 번호, 본체 IP, Jetson Ethernet 포트, 센서 전원 · 케이블. 정적 IP와 드라이버 설정에 필요하다.
카메라	OAK-D Lite FF 정확한 모델, USB 케이블/포트, 장착 방향과 실측 위치. 영상/깊이 프레임 검증에 필요하다.

항목	기록할 실제 값과 이유
구동부	RoboClaw 정확한 모델, 펌웨어, USB 포트, 모터 M1/M2와 좌우 대응, 엔코더 채널과 방향. 잘못 매핑하면 회전 방향이 뒤집힌다.
차체	실제 바퀴 반지름, 좌우 바퀴 중심 간 거리, 감속비, 바퀴 1회전당 엔코더 카운트. /cmd_vel 변환과 /odom 계산에 필요하다.
전원	배터리/BMS, 모터 컨트롤러 입력 전압, Jetson용 안정화 전원, LiDAR 전원, 퓨즈 및 물리적 비상정지 회로. 정격은 부품 실물 설명서로 확인한다.
ROS 네트워크	Jetson 호스트명/IP, 개발 PC IP, ROS_DOMAIN_ID, 사용할 유선/무선 인터페이스. 원격 관찰과 DDS 통신에 필요하다.

Gazebo의 TF 프레임 구조가 같아도 실제 장착 위치 · 바퀴 치수 · 엔코더 분해능이 같은지는 별도 확인한다. TF의 수치가 다르면 URDF를 실측값으로 갱신한다.

1. Jetson 펌웨어와 OS 설치

1. 보드와 저장장치 모델을 확인한다. NVIDIA 공식 Orin Nano 개발자 키트 빠른 시작의 해당 절차를 따른다. 현재 공식 절차는 JetPack 7.2.1의 Ubuntu 24.04 기반 이미지와 USB 설치 매체를 설명한다. ISO 파일을 목표 microSD에 그대로 굽는 절차가 아니다. 이미지 버전은 설치 당일 공식 페이지에서 다시 확인한다.
2. 새 보드의 UEFI/QSPI 펌웨어가 해당 이미지의 최소 버전을 만족하는지 확인한다. 구형 펌웨어라면 공식 펌웨어 갱신 절차를 먼저 따른다. 설치 과정은 목표 저장장치의 데이터를 지우므로 저장장치를 정확히 식별한다.
3. 설치 후 Jetson 터미널에서 확인한다.

```
uname -m                # aarch64 ■■
lsb_release -rs         # 24.04 ■■
cat /etc/nv_tegra_release
uname -r
df -h
```

4. NVIDIA 저장소와 드라이버가 잡혔는지 확인하고, 공식 JetPack 설정 안내에 따라 필요한 SDK를 설치한다.

```
sudo apt update
sudo apt install nvidia-jetpack
```

왜: ROS Jazzy의 공식 Ubuntu 패키지와 현재 Limbo의 기준 환경을 맞추고, Jetson GPU/카메라 사용자 공간 라이브러리의 버전을 OS와 일치시킨다. 일반 PC의 Ubuntu 24.04 ISO를 Jetson에 직접 설치하는 방식으로 진행하지 않는다.

2. ROS Jazzy와 Limbo를 Jetson에서 빌드

1. ROS 2 Jazzy Ubuntu 설치 지침에 따라 Ubuntu Noble arm64용 ROS 저장소와 ros-jazzy-ros-base 또는 ros-jazzy-desktop을 설치한다. 설치 페이지의 저장소 설정 명령은 최신 공식 지침을 그대로 따른다. 첫 확인:

```
source /opt/ros/jazzy/setup.bash
ros2 --help
printenv ROS_DISTRO
```

2. 저장소를 Jetson에 받고, 서브모듈을 포함해 소스를 준비한다. 이미 복사된 저장소가 있다면 그 저장소에서 동일하게 진행한다.

```
git clone --recurse-submodules https://github.com/jakemoti0n/final_team7.git ~/limbo
cd ~/limbo/limbo
```

3. rosdep으로 ROS/시스템 의존성을 확인하고 빌드한다. Jetson의 메모리에 맞춰 처음에는 순차 빌드한다.

```
sudo rosdep init                    # ■■ ■■ ■ ■ ■■; ■■ ■■■■■■ ■■
rosdep update
rosdep install --from-paths src --ignore-src -r -y
colcon build --symlink-install --executor sequential
source install/setup.bash
ros2 pkg list | grep '^limbo_'
```

실패한 패키지는 에러와 의존성 이름을 기록해 해결한다. x86 PC의 build/, install/, Python 가상환경을 Jetson에 복사하지 않는다. Python wheel, C++ 바이너리, 네이티브 라이브러리는 aarch64용으로 다시 설치/빌드해야 한다.

4. 사람 검출용 PyTorch/torchvision/Ultralytics는 별도 검증한다. 저장소 최상위 requirements.txt는 CPU PyTorch 인덱스를 사용하고, limbo/requirements.txt는 PC 전체 환경을 담은 긴 freeze 파일이므로 Jetson에 그대로 설치하지 않는다. 설치한 JetPack 버전에 맞는 NVIDIA Jetson PyTorch 설치 안내와 호환성 표를 확인한다. 해당 버전에서 제공되는 패키지/컨테이너 경로를 선택한 뒤 import torch, torch.cuda.is_available(), 실제 YOLO 모델의 이미지 1장 추론을 확인한다. GPU 설치가 바로 되지 않더라도 모터 · LiDAR · 카메라 기본 통합은 독립적으로 진행할 수 있다.

왜: 시뮬레이션 알고리즘은 ROS 메시지 인터페이스에 의존하지만, 드라이버와 Python/C++ 바이너리는 Jetson CPU 아키텍처 · JetPack 버전에 의존한다.

3. 전원과 통신 물리 연결: 장치별로 하나씩

하드웨어 담당자가 배선한 후, 멀티미터로 극성 · 전압 · 공통 접지 · 퓨즈 · 비상정지 동작을 확인하고 소프트웨어 담당자에게 연결표를 전달한다. 처음에는 바퀴가 바닥에 닿지 않도록 차체를 고정한다.

- Jetson: 공식 어댑터 또는 보드에 맞는 검증된 전원으로 단독 부팅한다. 배터리 전압을 임의로 Jetson 입력에 연결하지 않는다.
- MID-360S: 사용 설명서에 맞는 전원과 Ethernet 케이블을 사용한다. MID-360S는 Ethernet 커넥터에서 PoE로 전원 공급하는 장치가 아니다. 실제 구입한 케이블/분배기의 핀 배치는 모델과 설명서로 확인한다.
- OAK-D Lite FF: 데이터와 전원을 공급할 수 있는 USB 3 케이블/포트로 Jetson에 연결한다. 연결이 불안정하면 케이블, 포트, 전원 공급 가능한 허브를 점검한다. Luxonis USB 배포 안내를 따른다.
- RoboClaw: USB 통신만으로 모터 전원이 공급되지 않는다. 모터 전원, 엔코더, 비상정지 결선을 Basicmicro 시작 안내와 운용 요구사항에 맞춰 확인한다. 모터 전원 극성과 비상정지는 USB 연결 전에 검증한다.

Jetson에서 먼저 연결 상태를 확인한다.

```
lsusb
lsusb -t
ls -l /dev/serial/by-id/
nmcli device status
ip -4 addr
```

/dev/ttyACM0 같은 번호는 재부팅 · 재연결 시 바뀔 수 있다. 가능하면 /dev/serial/by-id/...의 안정적인 경로를 RoboClaw 설정에 사용한다. 장치 접근 권한이 없다면 해당 장치 파일의 그룹을 확인한 뒤 사용자 그룹을 설정하고 재로그인한다.

4. RoboClaw만으로 모터 · 엔코더 검증

1. Windows PC에서 Basicmicro Motion Studio로 RoboClaw 모델/펌웨어를 확인하고, 모터 채널 · 엔코더 채널 · 통신 모드를 설정한다. Linux USB 제어는 RoboClaw USB 제어 문서를 확인한다. 설정값은 파일/사진으로 남긴다.
2. 바퀴를 띄우고 좌/우 모터를 한쪽씩 아주 낮은 속도로 돌린다. + 명령이 로봇 전진 방향인지, 각 바퀴의 엔코더 카운트가 어느 방향으로 증가하는지 기록한다.
3. 모터 컨트롤러에서 읽은 엔코더 카운트와 실제 바퀴 1회전을 비교해 바퀴 1회전당 카운트를 구한다. 감속기와 quadrature 집계 배수를 포함한다.
4. 통신 끊김/명령 중단 시 모터가 멈추는 타임아웃과 물리적 비상정지를 실제로 시험한다. 소프트웨어 드라이버에도 명령 watchdog을 구현한다.

왜: 좌우 · 부호 · 엔코더 해석이 틀리면 Nav2가 정지 명령이나 선회 명령을 내도 로봇이 반대로 움직인다. 이 단계에서는 Nav2를 실행하지 않는다.

5. MID-360S Ethernet과 포인트클라우드

1. LiDAR 시리얼 번호의 끝 두 자리로 공장 IP를 확인한다. MID-360S 설명서의 기본 주소 규칙은 192.168.1.1XX이다. 실제 센서가 이미 재설정되었을 수도 있으므로 현장 IP를 확인한다.
2. Jetson의 LiDAR와 연결된 Ethernet 인터페이스에 같은 서브넷의 미사용 정적 주소를 준다. 예: 센서가 192.168.1.1XX/24면 Jetson을 192.168.1.50/24로 설정한다. Wi-Fi와 IP 대역이 겹치지 않게 한다. 설정 전후 nmcli device status, ip -4 addr, ping <센서-IP>로 확인한다. ping에 응답하지 않더라도 LiDAR 전용 진단 도구와 UDP 패킷 수신 여부를 추가 확인한다.
3. 공식 livox_ros_driver2의 MID-360S용 설정 파일인 MID360s_config.json에서 host_ip를 Jetson 유선 IP로, lidar_configs.ip를 실제 센서 IP로 설정한다. README의 SDK 의존성 · Jazzy 빌드 절차를 따른다. MID-360S와 Jazzy 지원은 드라이버 변경 이력을 확인한다.
4. 센서 드라이버만 실행하고 실제 발행 토픽/타입/주기와 header.frame_id, stamp를 확인한다. 드라이버 출력 이름이 Limbo의 /mid360/points와 다르면 remap 또는 변환 노드를 둔다.

```
ros2 topic list
ros2 topic info /mid360/points
ros2 topic hz /mid360/points
ros2 topic echo /mid360/points --once
```

드라이버 기본 출력이 다른 이름이면 위 명령의 토픽을 실제 출력 이름으로 바꾼다. 센서가 보인다는 것만으로 충분하지 않다. PointCloud2를 RViz에서 보고 바닥/벽이 실제 방향과 일치하며, 로봇 자체 포인트를 제거하는 필터가 동작하는지 확인한다.

6. OAK-D Lite FF 영상 · 깊이 · 포인트클라우드

1. lsusb, lsusb -t에서 장치와 USB 연결 속도를 확인한다. 필요한 udev 권한을 Luxonis USB 안내대로 설정하고 OAK Viewer 등으로 장치 단독 동작을 확인한다.
2. DepthAI ROS 드라이버의 Jazzy용 버전을 실제 Jetson arm64 배포 가능 여부와 함께 확인해 설치한다. ROS v3 패키지에는 _v3 접미사가 붙는 경우가 있으므로 설치된 패키지와 launch 이름을 ros2 pkg list로 확인한다.
3. 컬러 영상, 컬러 프레임에 정렬된 깊이, 해당 CameraInfo, 포인트클라우드를 발행하도록 설정한다. 실물 드라이버의 출력 이름을 다음 Limbo 입력 계약에 remap한다.

Limbo 입력	메시지/확인 내용
/rgbd_camera/image	sensor_msgs/Image, 컬러 영상
/rgbd_camera/depth_image	sensor_msgs/Image, 컬러 영상과 정렬된 깊이
/rgbd_camera/camera_info	sensor_msgs/CameraInfo, 깊이 계산에 대응하는 내부 파라미터
/rgbd_camera/points	sensor_msgs/PointCloud2, Nav2 장애물 · 충돌 감시 입력
rgbd_camera_optical_link	영상과 깊이의 광학 프레임. TF가 실제 카메라 설치 위치를 나타냄

4. 깊이 이미지 encoding과 단위를 확인한다. Limbo의 사람 위치 계산은 미터 단위 깊이를 기대한다. OAK 출력이 16UC1 밀리미터라면 중간 노드에서 미터로 변환하거나 사람 검출 코드에 명시적으로 단위 처리를 넣어야 한다. 영상/깊이 /CameraInfo의 header.stamp 차이도 확인한다. 현재 검출기 동기화 허용 폭은 약 0.03초다.

```
ros2 topic list
ros2 topic info /rgbd_camera/image
ros2 topic info /rgbd_camera/depth_image
ros2 topic info /rgbd_camera/camera_info
ros2 topic info /rgbd_camera/points
ros2 topic hz /rgbd_camera/image
ros2 topic echo /rgbd_camera/depth_image --once
```

왜: RGB 영상만 들어와도 사람 검출기가 시작될 수 있지만, 깊이 단위나 정렬이 틀리면 사람 거리와 지도 좌표가 잘못된다.

7. Limbo 저장소에서 필요한 변경

시뮬레이션 파일을 바로 덮어쓰기보다 실물용 launch와 설정 파일을 별도로 추가하면 두 환경을 비교해 검증할 수 있다. 아래는 현재 저장소를 기준으로 한 구현 목록이다.

파일/영역	해야 할 변경	이유/완료 기준
limbo/src/limbo_bringup/launch/start_simulation.launch.py	이를 복사 · 참고하여 start_robot.launch.py 작성. Gazebo, ros_gz_bridge, ground truth, 시뮬레이션 전용 노드를 제외하고 실제 드라이버와 robot_state_publisher, 필터, EKF, 위치추정, Nav2를 연결. 타이머 4/6/8/11/14초 대신 센서/TF/lifecycle 준비 상태를 확인.	부팅 속도가 달라도 선행 데이터 없이 Nav2가 시작되지 않음.
새 RoboClaw ROS 노드	/cmd_vel (geometry_msgs/Twist) 구독. 선속도 v, 각속도 ω, 바퀴 반지름 r, 바퀴 중심 거리 b로 ω_L=(v-ωb/2)/r, ω_R=(v+ωb/2)/r 계산 후 컨트롤러 단위로 변환. 엔코더에서 /odom (nav_msgs/Odometry) 발행. 포트 재연결/명령 타임아웃/모터 정지 구현.	전진 · 회전 부호와 정지 동작이 실제 바퀴와 일치. header.stamp, odom/base_footprint 프레임, twist covariance가 유효함.
IMU 드라이버	실제 IMU를 /imu (sensor_msgs/Imu)로 발행. BOM의 IMU 모듈/연결방식과 드라이버를 확인하고 축 · 단위 · 중력 · yaw-rate 부호를 검증. 아직 IMU가 없다면 EKF 구성과 최초 주행 가능 범위를 별도로 결정.	EKF가 기대하는 각속도가 들어오고 정지 시 값이 안정됨.
limbo/src/limbo_navigation/config/ekf.yaml	실물용 설정에서 use_sim_time: false; /odom의 전진속도와 /imu의 각속도 · 공분산을 실제 데이터에 맞게 조정.	EKF가 odom → base_footprint 동적 TF를 한 곳에서 발행.
limbo/src/limbo_description/urdf/limbo.urdf.xacro	프레임 구조는 유지하고 실측 wheel radius/separation 및 센서 장착 변환을 비교 · 수정.	TF 트리에 누락/중복 동적 변환이 없고 센서 관측 위치가 실물과 일치.
limbo/src/limbo_navigation/config/nav2_params.yaml	실물용 복사본을 만들어 use_sim_time: false, 실제 footprint, 속도/가속도/장애물 거리, 센서 토픽을 설정. ToF의 tof_mark 제거. 충돌 감시 입력과 timeout을 실제 토픽 주기에 맞춤.	costmap과 collision monitor에 LiDAR/OAK 장애물이 표시됨.
limbo/src/limbo_navigation/config/amcl.yaml 및 지도	use_sim_time: false, 실제 환경에서 만든 지도와 실제 초기 위치를 사용. 새로운 공간에서는 먼저 SLAM으로 지도 작성.	RViz에서 로봇 pose와 실제 위치 · 방향이 일치.
limbo/src/limbo_perception/limbo_perception/person_detector.py	OAK의 토픽 · 깊이 단위 · 영상/깊이 정렬을 확인하고 필요한 변환 적용. Jetson용 모델 실행 경로/성능도 검증.	사람이 없는/있는 단일 프레임에서 예측과 거리 계산이 합리적.
ToF 관련 노드와 행동 트리	첫 실물 launch에서 tof_node를 시작하지 않는다. ToF가 없으면 /person_blocking, /person_approaching이 생성되지 않으므로 이를 기다리는 사람 회피/대기 경로를 임시로 비활성화하거나 LiDAR/OAK 기반 신호로 대체. 두 행동 트리의 <BackUp> 동작도 현재 운영 방침과 맞는지 검토.	없는 센서를 무한히 기다리지 않고, 후진 금지 정책이 있다면 실제 BT도 이를 지킴.

중요한 토픽 경계: 현재 시뮬레이션의 bridge.yaml이 Gazebo와 ROS를 이어 준다. 실물에서는 각 드라이버가 같은 ROS 메시지를 직접 제공하거나 remap/변환 노드가 제공해야 한다. /cmd_vel을 발행하는 Nav2와 이를 받는 RoboClaw 브리지 사이에 명령 경로가 하나만 있어야 한다.

/odom 발행자와 odom → base_footprint TF 발행자는 구별한다. 현재 EKF가 해당 TF를 발행하도록 설정되어 있으므로 RoboClaw 노드가 같은 TF를 추가 발행하면 안 된다. map → odom은 AMCL 또는 SLAM 중 현재 사용하는 한 노드가 담당한다.

8. 소프트웨어 통합과 관찰 순서

아래 순서를 각자 별도 터미널에서 실행하고, 매 단계에서 데이터가 정상인 것을 보고 다음 단계로 진행한다.

1. 모든 터미널에서 같은 ROS 환경을 source하고 ROS_DOMAIN_ID를 맞춘다. Jetson 내부 단일 컴퓨터 시험이면 기본값으로 시작해도 된다. 원격 PC에서 ROS를 볼 경우 두 기기의 네트워크와 DDS discovery를 확인한다.
2. ros2 param get /<노드명> use_sim_time으로 실물 노드가 모두 false인지 확인한다. Jetson 시간과 센서 메시지 stamp가 현재 시간과 일치해야 한다. 멀티 컴퓨터라면 NTP/chrony 등으로 시계를 동기화한다.
3. robot_state_publisher와 실물 센서 드라이버를 실행한다. ros2 run tf2_ros tf2_echo base_footprint <센서프레임>으로 고정 TF를 확인한다.
4. RoboClaw 브리지(모터 출력은 아직 차단하거나 바퀴를 띄움)를 실행하고 /odom, /imu의 타입 · frame · stamp · 속도 부호를 확인한다. EKF를 실행한 뒤 ros2 run tf2_ros tf2_echo odom base_footprint로 이동에 따른 변환을 확인한다.
5. LiDAR/OAK 관측을 RViz에서 확인한다. 포인트가 회전 · 주행에 따라 로봇 프레임에 고정된 올바른 방향으로 보이는지 확인한다. Nav2 local/global costmap과 collision monitor의 입력 토픽이 실제로 들어오는지 확인한다.
6. SLAM으로 실환경 지도를 만들거나 이미 검증한 지도를 로드한다. AMCL의 초기 위치는 RViz 등에서 로봇의 실제 시작 위치로 지정한다. 시뮬레이션의 (0,0,0)을 기계적으로 사용하지 않는다.
7. Nav2 lifecycle 노드가 active이고, global/local costmap과 map → odom → base_footprint 연결이 유지되는지 본다. 사람 검출/행동 트리는 기본 주행이 안정된 다음 추가한다.

빠른 관찰 명령:

```
ros2 topic list
ros2 topic hz /odom
ros2 topic hz /imu
ros2 topic echo /cmd_vel --once
ros2 run tf2_ros tf2_echo odom base_footprint
ros2 lifecycle get /bt_navigator
```

9. 첫 주행: 작은 단계로 검증

1. 비상정지 담당자를 지정하고, 주변을 비운 뒤 바퀴를 띄운 상태에서 /cmd_vel 소량 명령으로 전진 · 정지 · 좌회전 · 우회전 방향을 확인한다. 명령이 사라지거나 프로세스가 종료되면 정지하는지 확인한다.
2. 넓은 평지에서 수동/teleop으로 저속 직진 짧은 거리와 제자리 회전을 실시한다. 실제 이동거리 · 각도와 /odom · EKF · TF를 비교해 바퀴 반지름, 바퀴 간 거리, 엔코더 부호를 보정한다.
3. 장애물 앞에서 LiDAR/OAK 포인트가 costmap과 collision monitor에 반영되는지 확인한다. 정지 거리는 실제 최고속도, 제동거리, 지연을 고려해 현장에서 측정한다.
4. 검증한 지도와 시작 pose를 사용해 가까운 자율 목표 하나를 보낸다. 경로 · 속도 · 회피 · 정지를 관찰한다. 이어서 여러 목표와 사람 검출을 추가한다.
5. 각 시험의 rosbag, launch 로그, 파라미터, 성공/실패 조건을 기록한다. 실패가 발생한 단계의 센서→TF→추정→계획→명령 순으로 원인을 좁힌다.

흔한 증상과 먼저 볼 곳

증상	우선 확인
ros2 topic list에 센서가 없음	USB/Ethernet 인식, 전원, IP/방화벽, 드라이버 launch, ROS_DOMAIN_ID
LiDAR 드라이버는 실행되지만 데이터 없음	MID-360S 실제 IP, Jetson 유선 IP, host_ip, 케이블/전원, UDP 포트
OAK 영상은 있으나 사람 거리가 이상함	깊이 encoding과 mm↔m, 컬러 정렬, CameraInfo, optical frame, stamp 차이
바퀴가 반대로 돎	M1/M2 좌우 대응, 모터 극성, 엔코더 부호, v/ω 변환
로봇이 움직여도 RViz pose가 고정됨	/odom · /imu 주기, EKF 설정, TF 중복, use_sim_time, stamp
Nav2가 시작했지만 목표로 가지 않음	map/AMCL 초기 pose, costmap 장애물, lifecycle 상태, collision monitor 명령 경로
명령이 멈춰도 바퀴가 돎	RoboClaw 명령 timeout, 브리지 watchdog, 물리적 비상정지. 즉시 시험 중단
사람 대기 행동이 끝나지 않음	ToF를 뺀 상태에서 /person_blocking · /person_approaching 의존성이 남아 있는지 확인

완료 시 팀이 남겨야 할 산출물

- Jetson OS/JetPack/ROS 버전과 설치 명령 기록
- 실제 배선도, 전원 정격 확인표, 비상정지 시험 결과
- MID-360S IP · 설정 파일, OAK launch · 토픽 remap, RoboClaw 설정 백업
- 차체 실측값, 모터 · 엔코더 방향 및 보정값
- 실물용 launch/파라미터, 지도, 첫 주행 rosbag과 결과 기록

참고 자료

- NVIDIA Jetson Orin Nano 개발자 키트 빠른 시작
- ROS 2 Jazzy Ubuntu 설치
- Livox MID-360S 사용 설명서와 livox_ros_driver2
- Luxonis USB 안내와 DepthAI ROS 드라이버
- Basicmicro RoboClaw 시작 안내